

RECURSION

Prof. Madonna Lamin



Russian Doll

WHAT IS RECURSION?

- Learning recursion yields **maintainable, consistent, provably correct code**.
- Recursion is **cool** because it's both simple and powerful.
- Basically, it's a way to **solve tough problems** by using functions or procedures that refer back to themselves.
- So basically, recursion is when a function calls itself to solve a problem.

MANY ALGORITHMS TECHNIQUES ARE BASED ON RECURSION

1. Dynamic Programming

- Dynamic Programming involves solving complex problems by breaking them down into **simpler subproblems** and **storing** their solutions to avoid redundant computations.
- Recursion is often used to solve these subproblems, with techniques like memoization optimizing the process by storing previously computed results.
- Classic examples include the Fibonacci sequence and shortest path problems.

MANY ALGORITHMS TECHNIQUES ARE BASED ON RECURSION

2. Backtracking

- Backtracking is a systematic approach to finding solutions by trying different paths and backtracking when a dead end is reached.
- Recursion is commonly employed in backtracking algorithms to explore all possible solutions through a tree-like structure.
- Problems such as N-Queens, Sudoku, and maze solving are typically tackled using backtracking techniques.

MANY ALGORITHMS TECHNIQUES ARE BASED ON RECURSION

3. Divide and Conquer (Binary Search, Quick Sort, and Merge Sort)

- Divide and Conquer involves breaking down a problem into smaller **subproblems**, solving them **recursively**, and **combining** their solutions.
- Recursion is fundamental to this approach, with algorithms like Binary Search, Quick Sort, and Merge Sort utilizing recursive techniques.
- Binary Search efficiently searches a sorted array by recursively dividing it in half.
- Quick Sort and Merge Sort are recursive sorting algorithms that divide the array and merge sorted subarrays, respectively, to achieve sorting.

MANY ALGORITHMS TECHNIQUES ARE BASED ON RECURSION

4. Tree Traversal

- Tree Traversal algorithms systematically **visit and process** all nodes in a tree data structure.
- Recursion simplifies the implementation of tree traversal algorithms like Depth-First Search (**DFS**) and Breadth-First Search (**BFS**).
- DFS explores as far as possible along each branch before backtracking, making it well-suited for recursive implementation.

MANY ALGORITHMS TECHNIQUES ARE BASED ON RECURSION

5. Sorting Algorithm

- Various sorting algorithms, such as Quick Sort, Merge Sort, and Heap Sort, utilize recursion to efficiently sort elements.
- Recursion enables these algorithms to divide the array into smaller subarrays recursively until they are individually sorted and then merge them back together.
- Quick Sort, for instance, partitions the array recursively based on a pivot element, while Merge Sort divides the array into halves and merges them in sorted order.

IMPORTANCE OF RECURSION

- Recursion is important because it gives us a **clever way to solve problems**.
- It breaks down big problems into smaller, more **manageable parts**, making our code easier to understand.
- We use recursion a lot for problems that have a **natural repetitive structure**, like sorting or working with trees.
- Plus, learning how to think recursively helps us become **better problem solvers**, not just in programming but also in other areas where we need to break down complex problems.
- And in computer science theory, **recursion is a big deal** because it helps us understand how computers work and how to measure how hard or easy a problem is to solve. So, basically, recursion is a **handy tool that helps us solve problems smarter and understand computers better**.

WE WILL USE RECURSION IN THIS UNIT TO:

1. Binary Search – Iterative and Recursive Approach
2. Factorial Function
3. Compute Fibonacci Sequence
4. Determine whether a word is palindrome
5. Compute powers of a number
6. Solve the ancient Towers of Hanoi problem.

FACTORIAL FUNCTION - ITERATIVE

Problem Statement: Given a number X, print its factorial.

To obtain the factorial of a number, it has to be multiplied by all the whole numbers preceding it. More precisely

$$X! = X \times (X - 1) \times (X - 2) \dots \times 3 \times 2 \times 1$$

Note: X is always a positive number.

$$\text{Factorial}(n) = \begin{bmatrix} 1 & \text{if } n = 0 \\ n \times (n - 1) \times (n - 2) \times \dots \times 3 \times 2 \times 1 & \text{if } n > 0 \end{bmatrix}$$

Iterative Factorial Algorithm Definition

Example 1:

Input: $X = 5$

Output: 120

Explanation: $5! = 5*4*3*2*1$

Example 2:

Input: $X = 3$

Output: 6

Explanation: $3!=3*2*1$

- Note that this definition is **iterative**.
- A repetitive algorithm is defined iteratively whenever the definition involves only the algorithm parameter(s) and not the algorithm itself.
- We can calculate the factorial(4) using:
 - $\text{factorial}(4) = 4 \times 3 \times 2 \times 1 = 24$

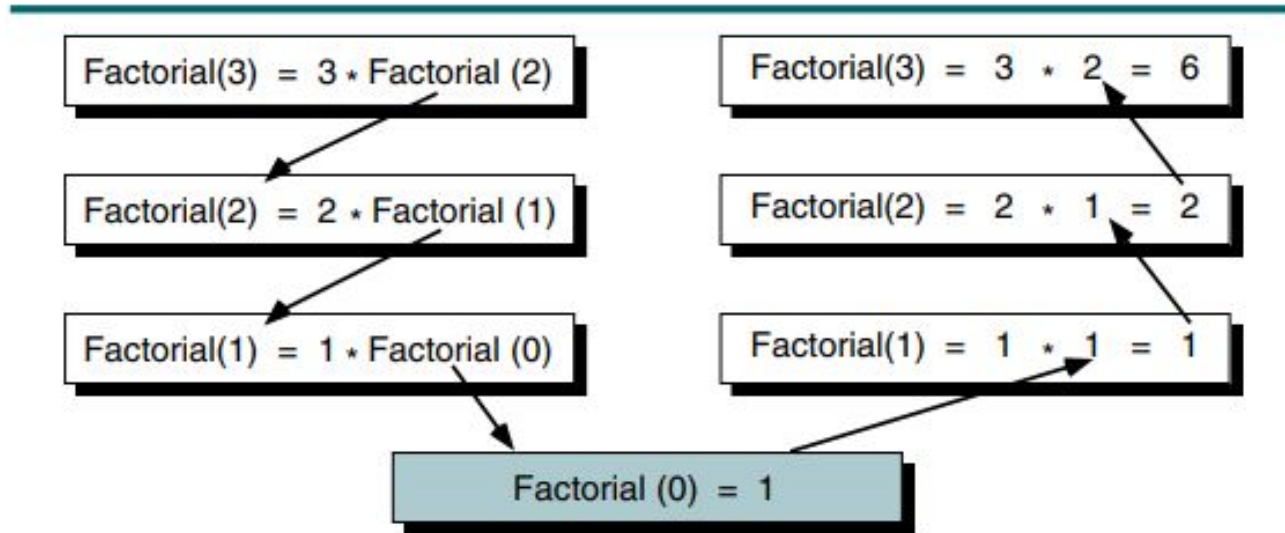
FACTORIAL - RECURSION DEFINED

- A repetitive algorithm uses **recursion** whenever the algorithm appears within the definition itself.
- For example, the factorial algorithm can be defined recursively

$$\text{Factorial } (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times (\text{Factorial } (n - 1)) & \text{if } n > 0 \end{cases}$$

Recursive Factorial Algorithm Definition

DECOMPOSITION OF FACTORIAL(3)



Factorial (3) Recursively

“Recursion is a repetitive process in which algorithm calls itself”

ITERATIVE SOLUTION

Iterative Factorial Algorithm

```
Algorithm iterativeFactorial (n)
Calculates the factorial of a number using a loop.
  Pre  n  is the number to be raised factorially
  Post n! is returned
1 set i to 1
2 set factN to 1
3 loop (i <= n)
  1 set factN to factN * i
  2 increment i
4 end loop
5 return factN
end iterativeFactorial
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

RECURSIVE SOLUTION

- The recursive solution does not need a loop; recursion is itself repetition.
- In the recursive solution, we let the factorial algorithm call itself, each time with a different set of parameters.

Recursive Factorial

Algorithm recursiveFactorial (n)

Calculates factorial of a number using recursion.

Pre n is the number being raised factorially

Post n! is returned

1 if (n equals 0)

1 return 1

2 else

1 return (n * recursiveFactorial (n - 1))

3 end if

end recursiveFactorial

Time Complexity: $O(n)$ since the function is being called n times, and for each function, we have only one printable line that takes $O(1)$ time, so the cumulative time complexity would be $O(n)$

Space Complexity: $O(n)$, in the worst case, the recursion stack space would be full with all the function calls waiting to get completed and that would make it $O(n)$ recursion stack space.

- If you compare the iterative and recursive versions of factorial, you should be immediately struck by how much **simpler** the code is in the recursive version.
- First, there is **no loop**.
- The recursive version consists of a **simple selection statement** that returns either the value 1 or the product of the two values, one of which is a call to a factorial itself.

```
program factorial
1 factN = recursiveFactorial(3)
2 print (factN)
end factorial
```

6

```
Algorithm recursiveFactorial (n)
1 if (n equals 0)
  1 return 1
2 else
  1 return (n x recursiveFactorial (n - 1))
3 end if
end recursiveFactorial
```

2

```
Algorithm recursiveFactorial (n)
1 if (n equals 0)
  1 return 1
2 else
  1 return (n x recursiveFactorial (n - 1))
3 end if
end recursiveFactorial
```

1

```
Algorithm recursiveFactorial (n)
1 if (n equals 0)
  1 return 1
2 else
  1 return (n x recursiveFactorial (n - 1))
3 end if
end recursiveFactorial
```

1

```
Algorithm recursiveFactorial (n)
1 if (n equals 0)
  1 return 1
2 else
  1 return (n x recursiveFactorial (n - 1))
3 end if
end recursiveFactorial
```

Calling a Recursive Algorithm

LIMITATIONS OF RECURSION

- Recursion works best when the algorithm uses a **data structure** that naturally supports recursion.
 - For example: **Trees** are naturally recursive structure and recursion works well with them.
- In other cases, the **algorithm** is naturally suited to recursion
 - For example: The **binary search algorithm** lends itself to a natural recursive algorithm, as does the Towers of Hanoi algorithm.
- On the other hand, not all looping algorithms can or should be implemented with recursion.
- Recursive solutions may involve **extensive overhead** (both time and memory) because they use calls.
 - Each call takes time to execute
- A recursive algorithm therefore generally runs more slowly than its non-recursive implementation.

You should NOT use recursion if the answer to any of the following questions is NO:

1. Is the algorithm or data structure suited to recursion?
2. Is the recursive solution shorter and more understandable?
3. Does the recursive solution run within acceptable time and space limits?

- Each time we make a call we use up some of our memory allocation.
 - If the recursion is deep - that is, if there are many recursive calls- we may run out of memory.
- Because of the the time and memory overhead, algorithms such as factorial are better developed iteratively if large numbers are involved.
- As a general rule, recursive algorithms should be used only when their **efficiency is logarithmic**.

RECURSIVE EXAMPLES

1. Binary Search
2. Factorial of a number
3. Fibonacci numbers
4. Greatest Common Divisor (GCD)
5. Determine whether a word is palindrome
6. Compute powers of a number
7. Prefix to Postfix conversion
8. Towers of Hanoi

FIBONACCI NUMBERS

- Fibonacci numbers are named after **Leonardo Fibonacci**, an **Italian mathematician** who lived in the early **thirteenth century**.
- In this series, each number is the sum of the previous two numbers.
- The first few Fibonacci series are:
 - 0,1,1,2,3,5,8,13,21,34,...

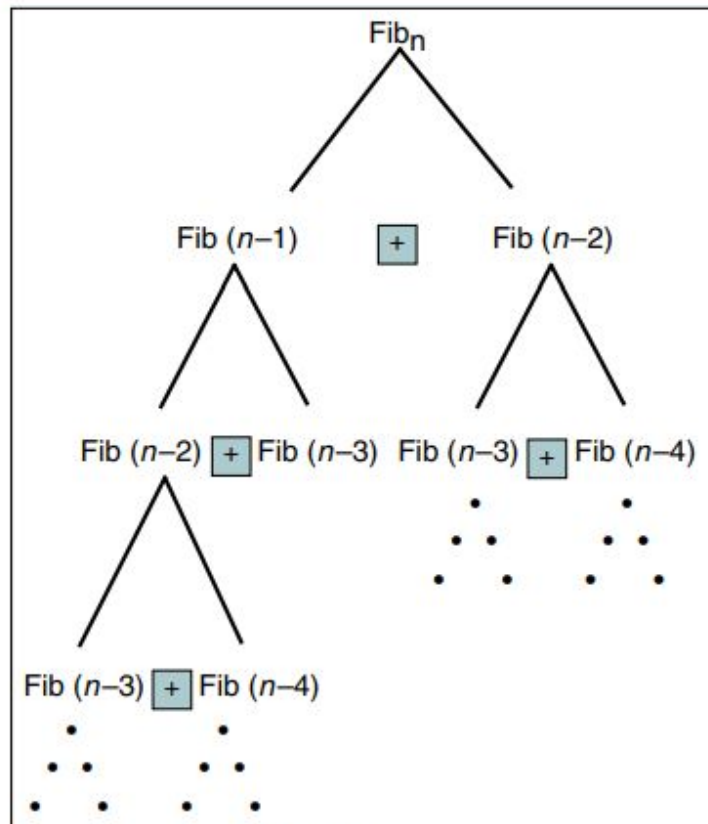
DESIGN

- To start a Fibonacci series, we need to know the first two numbers.
- These are 0 and 1. These are the base cases.

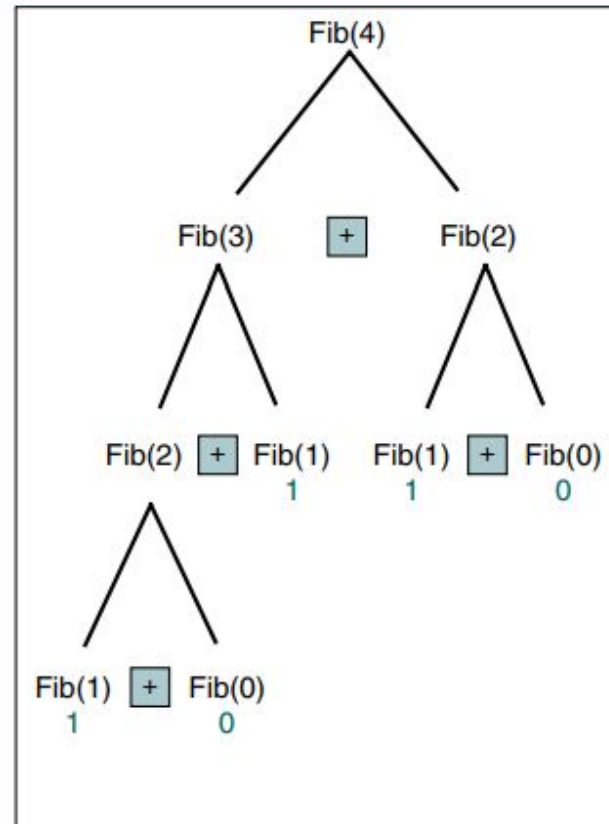
$$\text{Fibonacci}(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ \text{Fibonacci}(n - 1) + \text{Fibonacci}(n - 2) & \text{otherwise} \end{cases}$$

Fibonacci Numbers Recursive Definition

FIBONACCI(4)



(a) $\text{Fib}(n)$



(b) $\text{Fib}(4)$

Fibonacci Numbers

TIME AND SPACE COMPLEXITY OF FIBONACCI SEQUENCE USING RECURSION

- For `fibonacci(n)`, it calls `fibonacci(n-1)` and `fibonacci(n-2)`.
- Each of those again makes two recursive calls, and so on.
- This forms a binary recursion tree.

The number of nodes in this tree grows exponentially, approximately **2^n** .

So:

Time Complexity = **$O(2^n)$** $\rightarrow$ Exponential Time (due to redundant computation)

The space complexity comes from the recursive call stack.

- At most, the recursion goes down n levels (from `fibonacci(n)` to `fibonacci(0)`).
- So, the maximum depth of recursion = n .
- Each recursive call uses stack memory.

Therefore:

Space Complexity = $O(n)$

```
int recursiveFib(int n){
    if(n == 0 || n == 1)
        return n;
    else
        return recursiveFib(n-1)+ recursiveFib(n-2);
}

void printFib(int terms) {
    for (int i = 0; i < terms; i++) {
        printf("%d ", recursiveFib(i));
    }
    printf("\n");
}
```

GREATEST COMMON DIVISOR (GCD)

- A common mathematics function is to determine the GCD for two numbers.
 - For example, the greatest common divisor for 10 and 25 is 5

GCD DESIGN

- We use the **Euclidean algorithm** to determine the greatest common divisor between two non-negative integers.
- Given two integers, a and b , the greatest common divisor is recursively found using the formula.

$$\text{gcd}(a, b) = \begin{cases} a & \text{if } b = 0 \\ b & \text{if } a = 0 \\ \text{gcd}(b, a \bmod b) & \text{otherwise} \end{cases}$$

Greatest Common Divisor Recursive Definition

- The Euclidean Division Algorithm is a method used in mathematics to find the Greatest Common Divisor (GCD) of two numbers.
- It is based on **Euclid's Division Lemma**.
- In this algorithm we repeatedly divide and find remainders until the remainder becomes zero.
- According to Euclidean Division Algorithm, two positive integers a and b , where $a = bq + r$, a common divisor of a and b is also a common divisor of b and r , and vice versa,
- $\text{GCD}(a,b) = \text{GCD}(b,r)$

Find GCF or GCD using the Euclidean Algorithm

Example:

Find GCD of 12 and 30

$$30 \div 12 = 2 \text{ remainder } 6$$


$$12 \div 6 = 2 \text{ remainder } 0$$

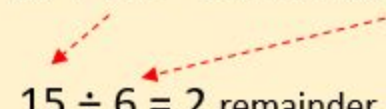
GCD

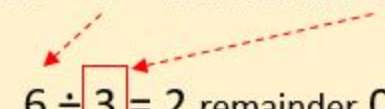
The GCD of 12 and 30 is 6

Find GCD of 123 and 36

$$123 \div 36 = 3 \text{ remainder } 15$$


$$36 \div 15 = 2 \text{ remainder } 6$$


$$15 \div 6 = 2 \text{ remainder } 3$$


$$6 \div 3 = 2 \text{ remainder } 0$$

GCD

The GCD of 123 and 36 is 3

Example 1: Find the GCD of 252 and 105

Solution:

Divide 252 by 105:

$$252 = 105 \times 2 + 42 \text{ (Quotient} = 2, \text{ Remainder} = 42)$$

Replace a with 105 and b with 42, then divide again, since $GCD(252, 105) = GCD(105, 42)$:

$$105 = 42 \times 2 + 21 \text{ (Quotient} = 2, \text{ Remainder} = 21)$$

Replace a with 42 and b with 21, then divide again, since $GCD(105, 42) = GCD(42, 21)$:

$$42 = 21 \times 2 + 0 \text{ (Quotient} = 2, \text{ Remainder} = 0)$$

Stop here because the remainder is now 0 and $GCD(42, 21) = GCD(21, 0)$.

And $GCD(21, 0) = 21$

then, $GCD(252, 105) = GCD(105, 42) = GCD(42, 21) = GCD(21, 0) = 21$

$GCD(252, 105) = 21$.

Example 2: Find the GCD of 360 and 96

Solution:

Divide 360 by 96:

$$360 = 96 \times 3 + 72 \text{ (Quotient} = 3, \text{ Remainder} = 72)$$

Replace a with 96 and b with 72, then divide again, since $\text{GCD}(360, 96) = \text{GCD}(96, 72)$:

$$96 = 72 \times 1 + 24 \text{ (Quotient} = 1, \text{ Remainder} = 24)$$

Replace a with 72 and b with 24, then divide again, since $\text{GCD}(96, 72) = \text{GCD}(72, 24)$:

$$72 = 24 \times 3 + 0 \text{ (Quotient} = 3, \text{ Remainder} = 0)$$

Stop here because the remainder is now 0 and $\text{GCD}(72, 24) = \text{GCD}(24, 0)$.

$$\text{And } \text{GCD}(24, 0) = 24$$

$$\text{then, } \text{GCD}(360, 96) = \text{GCD}(96, 72) = \text{GCD}(72, 24) = \text{GCD}(24, 0) = 24$$

$$\text{GCD}(360, 96) = 24$$

Practice Problem Based on Euclid's Division Algorithm

Question 1. Find the GCD of 120 and 45 using the Euclidean Division Algorithm.

Question 2. Find the GCD of 144 and 60 using the Euclidean Division Algorithm.

Question 3. Find the GCD of 462 and 192 using the Euclidean Division Algorithm.

Question 4. Find the GCD of 84 and 56 using the Euclidean Division Algorithm.

Answer:-

1. *GCD of 120 and 45 = 15*
2. *GCD of 144 and 60 = 12*
3. *GCD of 462 and 192 = 6*
4. *GCD of 84 and 56 = 28*

Euclidean Algorithm for Greatest Common Divisor

Algorithm gcd (a, b)

Calculates greatest common divisor using the Euclidean algorithm.

Pre a and b are positive integers greater than 0

Post greatest common divisor returned

1 if (b equals 0)

1 return a

2 end if

3 if (a equals 0)

2 return b

4 end if

5 return gcd (b, a mod b)

end gcd

HOW MANY STEPS DOES IT TAKE? (TIME COMPLEXITY)

Think like this:

- Each step makes the numbers smaller.
- In fact, the remainder gets smaller very quickly.

Example:

- From (48, 18) → we went to (18, 12) → then (12, 6) → then (6, 0).
- Just 3 steps and we got the answer!

Mathematicians proved that the slowest case happens when the numbers are consecutive Fibonacci numbers (like 21 and 34).

Even then, the number of steps is about logarithmic in size of the smaller number

Time Complexity = $O(\log(\min(a,b)))$

HOW MANY STEPS DOES IT TAKE? (TIME COMPLEXITY)

Even then, the number of steps is about logarithmic in size of the smaller number

Time Complexity = **$O(\log(\min(a,b)))$**

This means:

- If the numbers are small -> very fast
- If the numbers are huge -> still fast, because logarithm grows very slowly.

HOW MUCH MEMORY DOES IT USE? (SPACE COMPLEXITY)

Two cases:

1. Recursive Version

- Every recursive call goes deeper until $b = 0$.
- Depth = number of steps = $O(\log(\min(a, b)))$.
- So space = $O(\log(\min(a, b)))$ (because of stack frames).

2. Iterative Version

- We don't use recursion, just a loop.
- Only a few variables are needed.
- So space = $O(1)$.

DETERMINE WHETHER A STRING IS PALINDROME

Problem Statement: Given a string, check if the string is palindrome or not. A string is said to be palindrome if the reverse of the string is the same as the string.

Example:

Example 1:

Input Str: "ABCDcba"

Output: Palindrome

Example 2:

Input Str: A man, a plan, a canal, Panama

Output: Palindrome

DETERMINE WHETHER A STRING IS PALINDROME

Problem Statement: Given a string, check if the string is palindrome or not. A string is said to be palindrome if the reverse of the string is the same as the string.

Example:

Example 3:

Input Str: Mr. Owl ate my metal worm

Output: Palindrome

Example 4:

Input Str: Do geese see God?

Output: Palindrome

DETERMINE WHETHER A STRING IS PALINDROME

Problem Statement: Given a string, check if the string is palindrome or not. A string is said to be palindrome if the reverse of the string is the same as the string.

Example:

Example 5:

Input Str: Was it a car or a cat I saw?

Output: Palindrome

Example 6:

Input Str: Do geese see God?

Output: Palindrome

DETERMINE WHETHER A STRING IS PALINDROME

Problem Statement: Given a string, check if the string is palindrome or not. A string is said to be palindrome if the reverse of the string is the same as the string.

Example:

Example 7:

Input Str: tattarrattat

Output: Palindrome

Example 8:

Input Str: Malayalam

Output: Palindrome

DETERMINE WHETHER A STRING IS PALINDROME

Problem Statement: Given a string, check if the string is palindrome or not. A string is said to be palindrome if the reverse of the string is the same as the string.

Example:

Example 9:

Input Str: Hi there

Output: Not Palindrome

Example 8:

Input Str: To be or not to be

Output: Not Palindrome

RECURSION IDEA FOR PALINDROMES

- Instead of reversing the string, we can compare characters from the start and end moving inward.
- Base Case:
 - If left index $\geq$ right index $\rightarrow$ string is palindrome.
- Recursive Case:
 - If characters at left and right are equal $\rightarrow$ recurse with left+1 and right-1.
 - If characters don't match $\rightarrow$ not a palindrome.
- Instead of reversing the string, we can compare characters from the start and end moving inward.
- Base Case:
 - If left index $\geq$ right index $\rightarrow$ string is palindrome.
- Recursive Case:
 - If characters at left and right are equal $\rightarrow$ recurse with left+1 and right-1.
 - If characters don't match $\rightarrow$ not a palindrome.

Since your examples also include spaces, punctuation, and case, we need to clean the string first.

RECURSIVE ALGORITHM

1. Preprocess the string:

- Convert to lowercase.
- Remove spaces and non-alphanumeric characters.

2. Define a recursive function `isPalindrome(str, left, right)`:

- If `left >= right` → return `True`.
- If `str[left] != str[right]` → return `False`.
- Else → call `isPalindrome(str, left+1, right-1)`.

3. Return result.

PSEUDOCODE

Algorithm RecursivePalindrome(str):

1. Convert str to lowercase
2. Remove all characters that are not alphabets or digits
3. Define function isPalindrome(s, left, right):
 - if left >= right:
 - return True
 - if s[left] != s[right]:
 - return False
 - return isPalindrome(s, left+1, right-1)
4. Call result = isPalindrome(cleanedStr, 0, length(cleanedStr)-1)
5. If result is True:
 - print "Palindrome"Else:
 - print "Not Palindrome"

EXAMPLE WALKTHROUGH

Input: "Mr. Owl ate my metal worm"

- Clean string → "mrowlatemymetalworm"
- Call `isPalindrome("mrowlatemymetalworm", 0, 18)`
 - Compare first and last → `m == m`
 - Compare second and second last → `r == r`
 - Keep recursing until indices cross → Base case reached.
- Returns True → Palindrome.

PALINDROME COMPLEXITY

1. Time Complexity

Suppose the cleaned string has length n .

- Each recursive call compares two characters (left and right).
- After each step, we reduce the problem size by 2 (move inward).
- So total recursive calls $\approx n/2 \rightarrow O(n)$ comparisons.

Time Complexity = **$O(n)$**

2. Space Complexity

Two factors contribute:

1. Recursion Call Stack

- Each recursive call adds a new frame to the stack.
- In the worst case, there are $n/2$ recursive calls $\rightarrow O(n)$ stack space.

2. Extra Space for Cleaned String

- We also store a cleaned version of the string $\rightarrow O(n)$ extra space.

So overall space complexity:

Space Complexity = **$O(n)$**

COMPUTE POWERS OF A NUMBER USING RECURSION

Problem Statement

We need to compute the power of a number using recursion.

That means, given a base number x and an exponent n , we want to calculate:

$$x^n = x \times x \times x \text{ (} n \text{ times)}$$

For example:

- $2^5 = 2 \times 2 \times 2 \times 2 \times 2 = 32$
- $3^3 = 3 \times 3 \times 3 = 27$

Key Idea of Recursion

- Every recursive solution needs:
 1. **Base Case** → when should recursion stop.
 2. **Recursive Step** → reduce the problem into a smaller subproblem.

Here:

- **Base Case:**
 - If exponent $n = 0$, then $x^0 = 1$.
 - If exponent $n = 1$, then $x^1 = x$.
- **Recursive Step:**
 - $x^n = x \times x^{n-1}$

So, instead of multiplying x n times, we let recursion handle the multiplication step by step.

Algorithm

1. Start with input values: base `x`, exponent `n`.
2. If `n == 0`, return `1` (anything raised to the power 0 is 1).
3. If `n == 1`, return `x`.
4. Otherwise, recursively call the function with `(x, n-1)` and multiply the result by `x`.
 - That is: `power(x, n) = x * power(x, n-1)`
5. Return the final result.

PSEUDOCODE

```
function power(x, n):  
    if n == 0:  
        return 1  
    else if n == 1:  
        return x  
    else:  
        return x * power(x, n - 1)
```

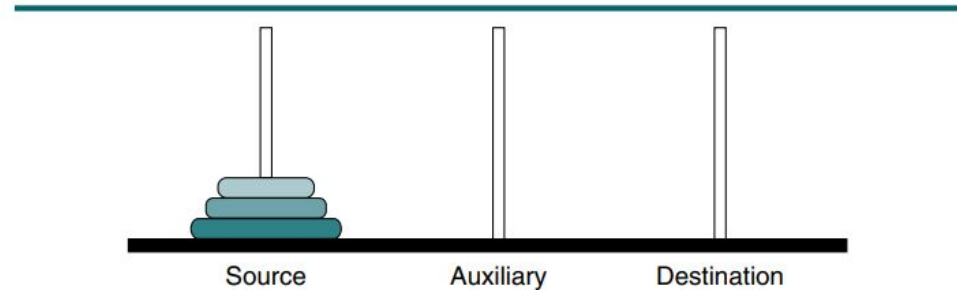
EXAMPLE WALKTHROUGH

```
power(2, 4)
= 2 * power(2, 3)
= 2 * (2 * power(2, 2))
= 2 * (2 * (2 * power(2, 1)))
= 2 * (2 * (2 * 2))
= 16
```

TOWERS OF HANOI

- The Towers of Hanoi is a **classic recursion problem** that is relatively easy to follow, is efficient, and uses no complex data structures.
- The Tower of Hanoi puzzle was invented by the **French mathematician Edouard Lucas in 1883**.
- According to the legend, the monks in a remote mountain monastery knew how to predict when the world would end.
- They had a set of **three diamond needles**.
- Stacked on the first diamond needle were 64 gold disks of decreasing size.
- The monks moved one disk to another needle each hour, subject to the following rules:

1. Only one disk could be moved at a time. A larger disk must never be stacked above a smaller one.
 2. One and only one **auxiliary needle** could be used for the intermediate storage of disks.
- The legend said that when all 64 disks had been transferred to the destination needle, the stars would be extinguished and the world would end.
 - Today we know that we need to have $2^{64} - 1$ moves to move all of the disks



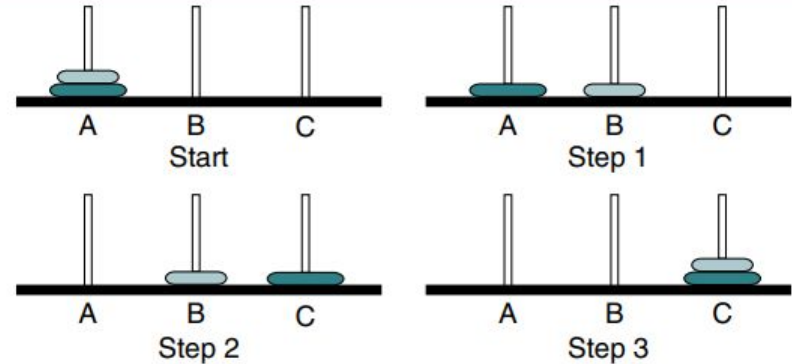
Towers of Hanoi—Start Position

- The problem is interesting for two reasons:
 - The **recursive solution is much easier** to code than the iterative solution would be, as is often the case with good recursive solutions.
 - Its solution pattern is different from the simple examples we have discussed above.
- As you study the towers solution, note that after each **base case**, we return to a **decomposition** of the **general case** for several steps.
- In other words, the problem is divided into several **subproblems**, each of which terminates with the base case, moving one disk.

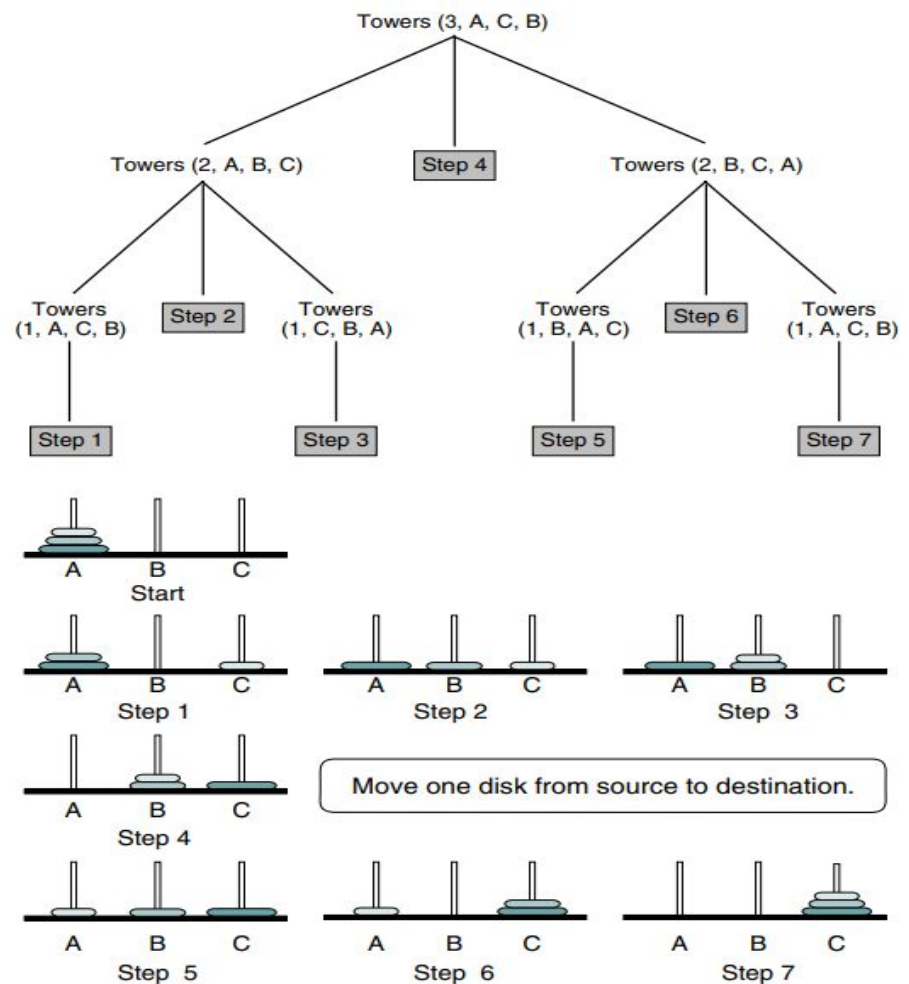
RECURSIVE TOWERS OF HANOI DESIGN

- To solve this problem, we must study the moves to see if we can find a pattern.
- We will use only **three disks** because we do not want the world to end.
- First, imagine we have only one disk to move. This is a very simple case.
- **Case 1:** Move one disk from **source** to **destination**
needle

- Now imagine that we have to move **two disks**.
- First we **move the top disk to the auxiliary needle**. Then we move the **second disk to the destination needle**. Finally we move the **first disk to the top of the second disk** on the destination needle.
- **Case 2:**
 - Move one disk to the auxiliary needle
 - Move one disk to the destination needle
 - Move one disk from the auxiliary to destination needle



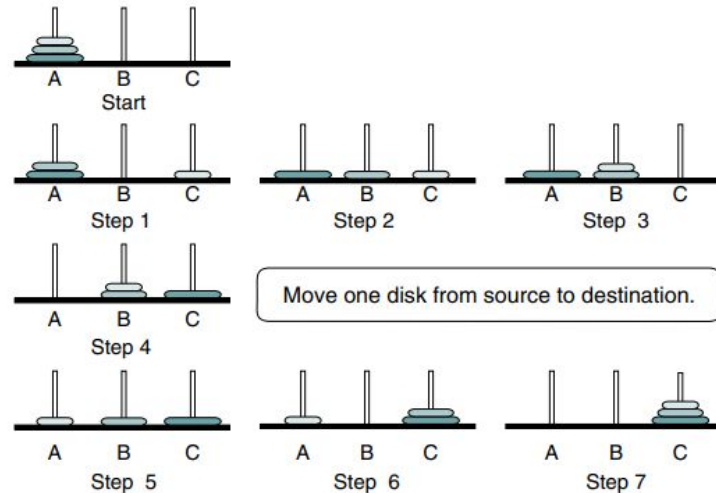
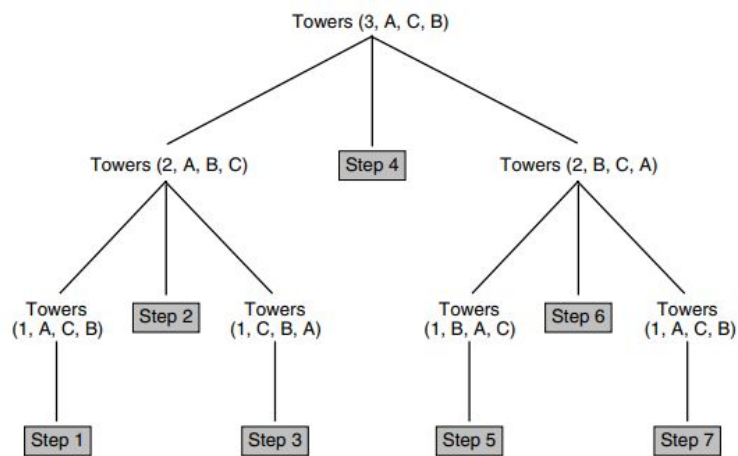
- The case for **three disks**.
- The **first three steps** move the top two disks from the source to auxiliary needle (Refer Case 2)
- In Step 4 we move the bottom disk to the destination needle.
- We now have one disk in place. This is example of Case 1.
- We then need three more steps to move the two disks on the auxiliary needle to the destination needle.



- The case for three disks.

Case 3:

1. Move two disks from source to auxiliary needle
2. Move one disk from source to destination needle
3. Move two disks from auxiliary to destination needle



We are now ready to generalize the problem.

1. Move **$n-1$ disks** from **source to auxiliary**, using destination.
2. Move **one disk** from **source to destination**. Base Case
3. Move **$n-1$ disks** from **auxiliary to destination**, using source General Case

- Our solution requires an algorithm with four parameters
 - The number of disks to be moved
 - The source needle
 - The destination needle
 - The auxiliary needle
- Using pseudocode the three moves in the generalization shown above are then:
 1. Call Towers($n-1$, source, auxiliary, destination)
 2. Move one disk from source to destination
 3. Call Towers($n-1$, auxiliary, destination, source)

- Study the second call above carefully
- After we complete the move of the first disk, the remaining disks are on the auxiliary needle.
- We need to move them from auxiliary needle to destination needle.
- In this case the original source needle becomes the auxiliary needle. (Remember that the positions of the parameters in the called algorithm are source, destination, and auxiliary. The calling algorithm must remember which of the three needles is the source and which is the destination for each call.)

Towers of Hanoi

Algorithm towers (numDisks, source, dest, auxiliary)

Recursively move disks from source to destination.

Pre numDisks is number of disks to be moved

source, destination, and auxiliary towers given

Post steps for moves printed

```
1 print("Towers: ", numDisks, source, dest, auxiliary)
2 if (numDisks is 1)
  1 print ("Move from ", source, " to ", dest)
3 else
  1 towers (numDisks - 1, source, auxiliary, dest, step)
  2 print ("Move from " source " to " dest)
  3 towers (numDisks - 1, auxiliary, dest, source, step)
4 end if
end towers
```

Calls:

Towers (3, A, C, B)

Towers (2, A, B, C)

Towers (1, A, C, B)

Towers (1, C, B, A)

Towers (2, B, C, A)

Towers (1, B, A, C)

Towers (1, A, C, B)

Output:

Move from A to C

Move from A to B

Move from C to B

Move from A to C

Move from B to A

Move from B to C

Move from A to C

Tracing Algorithm 2-7, Towers of Hanoi

THE JOSEPHUS PROBLEM

What is the Josephus Problem?

The Josephus Problem is a **famous puzzle in mathematics and computer science.**

It goes like this:

- Imagine **n people** standing in a circle
- You start counting them, one-by-one, around the circle.
- Every **kth-person is eliminated** (removed) from the circle.
- The counting continues from the next person after the one who was eliminated.
- This keeps going **until one person remains.**

The problem asks:

If you know **n (the total people)** and **k (the step)**, which position should you stand in to be the last survivor?

A SIMPLE EXAMPLE

Let's take $n = 5$ people standing in a circle:

Number them: 1, 2, 3, 4, 5

Suppose $k = 2$, meaning every 2nd person is removed.

Step by step elimination:

1. Start at 1 $\rightarrow$ count 2 $\rightarrow$ person 2 is eliminated.
Remaining: 1, 3, 4, 5
2. Next count starts at 3 $\rightarrow$ count 2 $\rightarrow$ person 4 is eliminated.
Remaining: 1, 3, 5
3. Start at 5 $\rightarrow$ count 2 $\rightarrow$ person 1 is eliminated.
Remaining: 3, 5
4. Start at 3 $\rightarrow$ count 2 $\rightarrow$ person 5 is eliminated.
Remaining: 3

So, if you were standing at position 3, you would survive!

WHY IS IT INTERESTING?

- At first, it looks like a simple game of counting, but the elimination keeps shifting in a tricky way.
- It's a great problem for understanding recursion (a way a function can call itself).
- It is also useful in computer science (like memory management, circular queues, or resource allocation problems).

THE RECURSIVE IDEA

The Josephus Problem can be solved with a formula:

$$J(n, k) = (J(n - 1, k) + k) \mod n$$

where:

- $J(n, k)$ = safe position when there are n people
- k = the step count
- Base case: $J(1, k) = 0$ (if there's only 1 person, that person survives)

(Note: The formula usually gives positions starting from 0. To convert to human-style counting (starting at 1), just add +1 at the end.)

The problem has the following recursive structure.

$josephus(n, k) = (josephus(n-1), k) \% n + 1$ and

$josephus(1, k) = 1$

After the first person (k th from the beginning is killed), $n - 1$ persons are left.

Make recursive call for $josephus(n-1, k)$ to get the position with $n - 1$ persons. But the position returned by $josephus(n-1, k)$ will consider the position starting from $k \% n + 1$. So make adjustments to the position returned by $josephus(n-1, k)$

Let's try $n = 5, k = 2$:

- $J(1, 2) = 0$
- $J(2, 2) = (0 + 2) \% 2 = 0$
- $J(3, 2) = (0 + 2) \% 3 = 2$
- $J(4, 2) = (2 + 2) \% 4 = 0$
- $J(5, 2) = (0 + 2) \% 5 = 2$

So, the survivor is at **position 2 (0-based)**.

In human counting (1-based), that's position 3- which matches our step-by-step elimination earlier.

SUMMARY

- The Josephus Problem is about finding the safest spot in a counting elimination game.
- It can be solved step-by-step OR with a neat recursive formula.
- It's a classic problem in algorithms and teaches you about recursion and modular arithmetic.

SUMMARY

- There are two approaches to writing repetitive algorithms : **iteration and recursion**
- **Recursion** is a repetitive process in which an algorithm calls itself.
- A repetitive algorithm uses recursion whenever the algorithm appears within the definition itself.
- When a program calls a subroutine, the current module **suspends** processing and the called subroutine takes over the control of the program. When the subroutine completes its processing and returns to the module that called it, the module **wakes up** and continues its processing.

- In a recursive algorithm, each call either solves part of the problem or reduces the size of the problem.
- The statement that solves the problem is known as the base case; every recursive algorithm must have a base case.
- The rest of the recursive algorithm is known as the general case.
- The general rule for designing a recursive algorithm is as follows:
 1. Determine the base case
 2. Determine the general case
 3. Combine the base case and the general case into an algorithm

EXERCISES

1. Consider the following algorithm

```
algorithm fun1 (x)
1  if (x < 5)
    1  return (3 * x)
2  else
    1  return (2 * fun1 (x - 5) + 7)
3  end if
end fun1
```

What would be returned if fun1 is called as

- a. fun1 (4)?
- b. fun1 (10)?
- c. fun1 (12)?

Solution:

`fun1(4)`

Here $x = 4$.

Since $4 < 5$, we use the base case:

$$\text{fun1}(4) = 3 * 4 = 12$$

Therefore $\text{fun1}(4) = 12$

Solution:

`fun1(10)`

Here $x = 10$.

Step 1 $10 \geq 5$

$$\text{fun1}(10) = 2 * \text{fun1}(10 - 5) + 7$$

$$\text{fun1}(5) = 2 * \text{fun1}(5 - 5) + 7$$

Now compute $\text{fun1}(0) = 3 * 0$ (since $0 < 5$)

Substitute back:

$$\text{fun1}(5) = 2 * 0 + 7 = 7$$

$$\text{fun1}(10) = 2 * 7 + 7 = 21$$

Solution:

fun1(12)

Here $x = 12$.

Step 1 $12 \geq 5$

$$\text{fun1}(12) = 2 * \text{fun1}(12 - 5) + 7$$

$$\text{fun1}(7) = 2 * \text{fun1}(7 - 5) + 7$$

Now compute $\text{fun1}(2) = 3 * 2$ (since $2 < 5$) = 6

Substitute back:

$$\text{fun1}(7) = 2 * 6 + 7 = 19$$

$$\text{fun1}(12) = 2 * 19 + 7 = 45$$

2. Consider the following algorithm

```
algorithm fun2 (x, y)
1  if (x < y)
    1  return -3
2  else
    1  return (fun2 (x - y, y + 3) + y)
3  end if
end fun2
```

What would be returned if fun2 is called as

- a. fun2 (2, 7)?
- b. fun2 (5, 3)?
- c. fun2 (15, 3)?

Solution

1. $\text{fun2}(2,7) = -3$
2. $\text{fun2}(5,3) = 0$
3. $\text{fun2}(15,3) = 6$

3. Consider the following algorithm

```
algorithm fun3 (x, y)
1  if (x > y)
    1  return -1
2  elseif (x equal y)
    1  return 1
3  else
    1  return (x * fun3 (x + 1, y))
4  end if
end fun3
```

What would be returned if fun3 is called as

- a. fun3 (10,4)?
- b. fun3 (4,3)?
- c. fun3 (4,7)?
- d. fun3 (0,0)?

Solution:

1. $\text{fun3}(10,4) = -1$

2. $\text{fun3}(4,3) = -1$

3. $\text{fun3}(4,7) = 120$

4. $\text{fun3}(0,0) = 1$

4. Ackerman's Number

Ackerman's number, used in mathematical logic, can be calculated using the formula shown in Figure 2-17. Write a recursive algorithm that calculates Ackerman's number. Verify your algorithm by using it to manually calculate the following test cases: $\text{Ackerman}(2, 3)$, $\text{Ackerman}(2, 5)$, $\text{Ackerman}(0, 3)$, and $\text{Ackerman}(3, 0)$.

$$\text{Ackerman}(m, n) = \begin{cases} n + 1 & \text{if } m = 0 \\ \text{Ackerman}(m - 1, 1) & \text{if } n = 0 \text{ and } m > 0 \\ \text{Ackerman}(m - 1, \text{Ackerman}(m, n - 1)) & \text{otherwise} \end{cases}$$

Ackerman Formula for Problem 6

5. Write a recursive algorithm that calculates and returns the length of a list.

6. Write a recursive algorithm that converts a string of numerals to an integer. For example, “43567” will be converted to 43567.

7. Write a recursive algorithm to add the first n elements of the series.

$$1 + 1/2 + 1/3 + 1/4 + 1/5 + \dots + 1/n$$

RECURSIVE ALGORITHM THAT CALCULATES AND RETURNS LENGTH OF A LIST

Key Idea of Recursion

- The length of a list is:

$1 + \text{length of the rest of the list (tail)}$

- **Base Case:** If the list is empty, its length is 0 .
- **Recursive Case:** Otherwise, return $1 + \text{length of (remaining list)}$.

RECURSIVE ALGORITHM

1. Input: A list `L`.
2. If `L` is empty $\rightarrow$ return `0`.
3. Otherwise $\rightarrow$ return `1 + length(tail(L))`.
 - Here, `tail(L)` means the list without its first element.

PSEUDOCODE

```
function listLength(L):  
    if L is empty:  
        return 0  
    else:  
        return 1 + listLength(L without first element)
```

EXAMPLE

Suppose $L = [10, 20, 30]$

`listLength([10,20,30])`

`= 1 + listLength([20,30])`

`= 1 + (1 + listLength([30]))`

`= 1 + (1 + (1 + listLength([])))`

`= 1 + (1 + (1 + 0))`

`= 3`

So the length is 3

RECURSIVE ALGORITHM THAT CONVERTS A STRING OF NUMERALS TO AN INTEGER

Write a recursive algorithm that converts a string of numerals to an integer. For example, “43567” will be converted to 43567.

KEY IDEA

A string of digits can be thought of as:

$$\text{"43567"} = (\text{"4356"} \times 10) + 7$$

That means:

- If the string has only **1 digit**, return its integer value.
- Otherwise, recursively convert the substring (excluding the last digit), multiply by 10, then add the last digit.

RECURSIVE ALGORITHM

1. Input: A string `s` of numerals.
2. If the length of `s` = 1 $\rightarrow$ return integer value of `s[0]`.
3. Otherwise:
 - Convert the substring `s[0 ... n-2]` recursively.
 - Multiply the result by 10.
 - Add the integer value of the last digit `s[n-1]`.
4. Return the result.

PSEUDOCODE

```
function stringToInt(S):  
    if length(S) == 1:  
        return int(S[0])  
    else:  
        return stringToInt(S[0 ... n-2]) * 10 + int(S[n-1])
```

EXAMPLE

```
stringToInt("43567")
```

```
= stringToInt("4356") * 10 + 7
```

```
= (stringToInt("435") * 10 + 6) * 10 + 7
```

```
= ((stringToInt("43") * 10 + 5) * 10 + 6) * 10 + 7
```

```
= (((stringToInt("4") * 10 + 3) * 10 + 5) * 10 + 6) * 10 + 7
```

```
= (((4 * 10 + 3) * 10 + 5) * 10 + 6) * 10 + 7
```

```
= 43567
```

RECURSIVE ALGORITHM TO ADD THE FIRST N ELEMENTS OF THE SERIES

Write a recursive algorithm to add the first n elements of the series. (Harmonic Series)

$$S(n) = 1 + 1/2 + 1/3 + 1/4 + 1/5 + \dots + 1/n$$

KEY IDEA OF RECURSION

- If $n = 1$, then $S(1) = 1$. (**Base Case**)
- Otherwise:

$$S(n) = S(n - 1) + \frac{1}{n}$$

RECURSIVE ALGORITHM

1. Input: integer n .
2. If $n = 1$, return 1 .
3. Otherwise, recursively compute $S(n-1)$ and add $1/n$.
4. Return the result.

PSEUDOCODE

```
function harmonicSum(n):  
    if n == 1:  
        return 1  
    else:  
        return harmonicSum(n - 1) + (1 / n)
```

EXAMPLE

```
harmonicSum(4)
= harmonicSum(3) + 1/4
= (harmonicSum(2) + 1/3) + 1/4
= ((harmonicSum(1) + 1/2) + 1/3) + 1/4
= ((1 + 1/2) + 1/3) + 1/4
= 1 + 0.5 + 0.333... + 0.25
= 2.083...
```

PREFIX TO POSTFIX CONVERSION

- An arithmetic expression can be represented in three different formats:
 - Infix
 - Prefix
 - Postfix
- In an infix notation, the operator comes between the two operands, the basic format of the algebraic notation.
- In postfix notation, the operator comes after its two operands.
- In prefix notation, the operator comes before its two operands.

Prefix:	+	A	B
Infix:	A	+	B
Postfix:	A	B	+

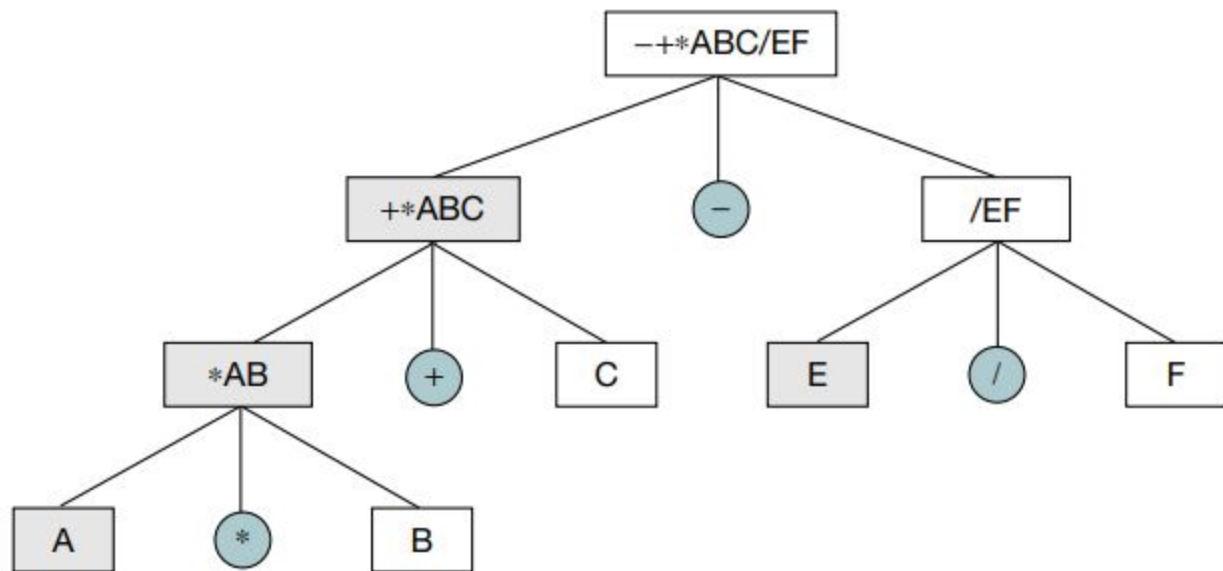
- Although some high-level languages use the infix and postfix notations, these expressions cannot be directly evaluated.
- They must be analyzed to determine the order in which the expressions are to be evaluated.
- One method to analyze them is to use recursion.

DESIGN

- Convert prefix to postfix format
- Basic concepts:
- Given the prefix expression: $*AB$, we convert it by moving the operator ($*$) after the operands (A and B).
- The postfix format is : $AB*$
- To keep the algorithm as simple as possible, we assume that each operand is only one character and that there are only four operators: add(+), subtract (-), multiply($*$) and divide (/)

- To convert a prefix expression we must find its operator and move it after the operands.
- By definition, the operator is always the first character in the prefix string. Following the operator are its two operands.

- As the expression grows in complexity, however, we can have multiple operators and operands in an expression.
 - For example: $-+*ABC/EF$
- This expression consists of several binary expressions combined into one complex expression.
- To parse it we begin at the left and work right until we isolate one binary expression, in this case $*AB$.
- Once we have a simple prefix expression, we can convert it to postfix and put it in the output.



› Decomposition of $-+*ABC/EF$

- The first step in designing a recursive algorithm is determining the base case. What is the base case?
 - The base case turns out to be **finding an operand**- in our example, any of the alphabetic characters.
- Because the operand can contain another expression, the general case is to **find the operator and the left and right operands** in the binary expression.
- We then concatenate them into one postfix expression.

- To find the left and right operands in the prefix expression, we need a second algorithm that determines the length of a prefix expression.
 - The length of an expression is the length of the first sub-expression plus the length of the second expression plus 1 for the operator.
- We also can use recursion for this algorithm.
- Because each operand is one character long, our base case is finding one operand.

Convert Prefix Expression to Postfix

```
Algorithm preToPostFix (preFixIn, postFix)
Convert a preFix string to a postFix string.
    Pre preFix is a valid preFixIn expression
    postFix is reference for converted expression
    Post postFix contains converted expression
1 if (length of preFixIn is 1)
    Base case: one character string is an operand
    1 set postFix to preFixIn
    2 return
2 end if
    If not an operand, must be an operator
3 set operator to first character of preFixIn
    Find first expression
4 set lengthOfExpr to findExprLen (preFixIn less first char)
5 set temp to substring(preFixIn[2, lengthOfExpr])
6 preToPostFix (temp, postFix1)
    Find second postFix expression
7 set temp to preFixIn[lengthOfExpr + 1, end of string]
8 preToPostFix (temp, postFix2)
    Concatenate postfix expressions and operator
9 set postFix to postFix1 + postFix2 + operator
10 return
end preToPostFix
```

- The algorithm is rather straightforward.
- The most difficult part is finding the prefix expression.
 - We find it by determining its length and then storing it in a temporary string.
- The notation in statement 5 indicates that the substring starting at the first location and through the second location is to be assigned to temp.
- Once we have identified a prefix expression, we recursively called the algorithm to convert it to postfix.

- Now let's look at the algorithm that determines the length of the prefix expression.
- This algorithm needs to return the length of the expression.
- Once again the base case is finding an operand.
- From the definition of the problem, we know that each operand is one character long.
- The minimum length of a prefix expression is therefore three: one for the operator and one for each operand.
- As we recursively decompose the expression, we continue to add the size of its combined expression until we have the total length.

Find Length of Prefix Expression

Algorithm findExprLen (exprIn)

Recursively determine the length of a prefix expression.

Pre exprIn is a valid prefix expression

Post length of expression returned

1 if (first character is operator)

General Case: First character is operator

Find length of first prefix expression

1 set len1 to findExprLen (exprIn + 1)

2 set len2 to findExprLen (exprIn + 1 + len2)

2 else

Base case--first char is operand

1 set len1 and len2 to 0

3 end if

4 return len1 + len2 + 1

end findExprLen

- We examine the first character of a string containing a prefix expression or a substring.
- If it is an operator, we recursively add the length of its two operands.
- The algorithm returns 1 (the base case) or the length of an expression in the prefix expression.